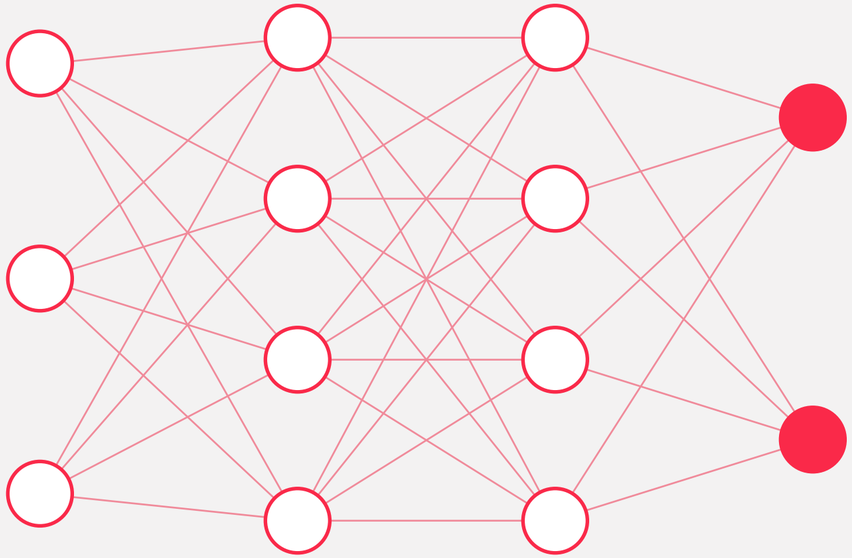
Agentic AI

Unidad 5 — Agentes de IA: razonamiento, memoria, herramientas y evaluación.



Contenido

01 Fundamentos de agentes (ReAct)

02 Memoria y estado

03 Múltiples herramientas

04 Grafos con LangGraph

05 RAG como herramienta

06 Sistemas multiagente

07 Evaluación de agentes

01

Fundamentos de Agentes (el patrón ReAct)

01

¿Qué le agrega un agente a un LLM?

Un LLM aislado solo produce texto a partir de texto. No puede consultar información actualizada ni ejecutar acciones

LLM

Solo texto → texto
Sin datos actuales
Sin acciones

\+

Herramientas

Funciones que puede invocar (calculadora, búsqueda, base de datos)

Ciclo de decisión

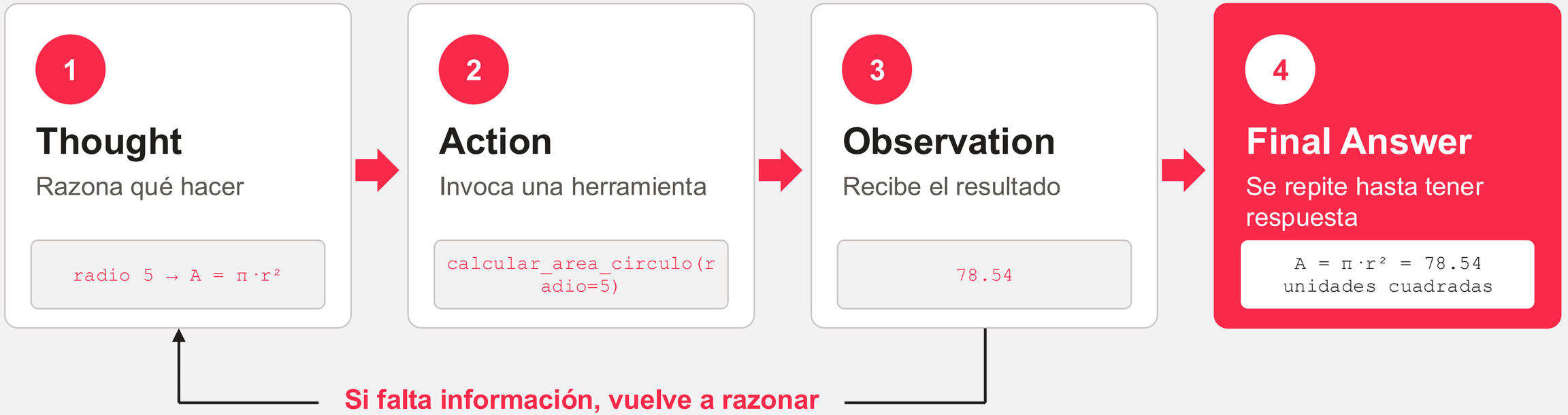
Decide en cada paso si ya puede responder o necesita usar una herramienta

Autonomía

El LLM elige qué herramienta usar y con qué argumentos, sin una secuencia programada de antemano

Agente = LLM + herramientas + ciclo de decisión + autonomía

El patrón ReAct: razonar y actuar en ciclos



Intercalar razonamiento entre cada acción permite que el agente replantee su estrategia si una herramienta no devolvió lo esperado, en vez de seguir ciegamente.

El ciclo ReAct, en una conversación real

1 Pregunta

Calcula el área de un círculo de radio 5 y explica por qué usas esa fórmula.

2 Action

```
calcular_area_circulo(radio=5)
```

El agente decide invocar la herramienta, en lugar de calcular "de memoria"

3 Observation

78.54

Resultado devuelto por la herramienta al agente

4 Final Answer

Para calcular el área de un círculo, utilizamos $A = \pi \cdot r^2$. El resultado es 78.54 unidades cuadradas...

02

Memoria y Estado

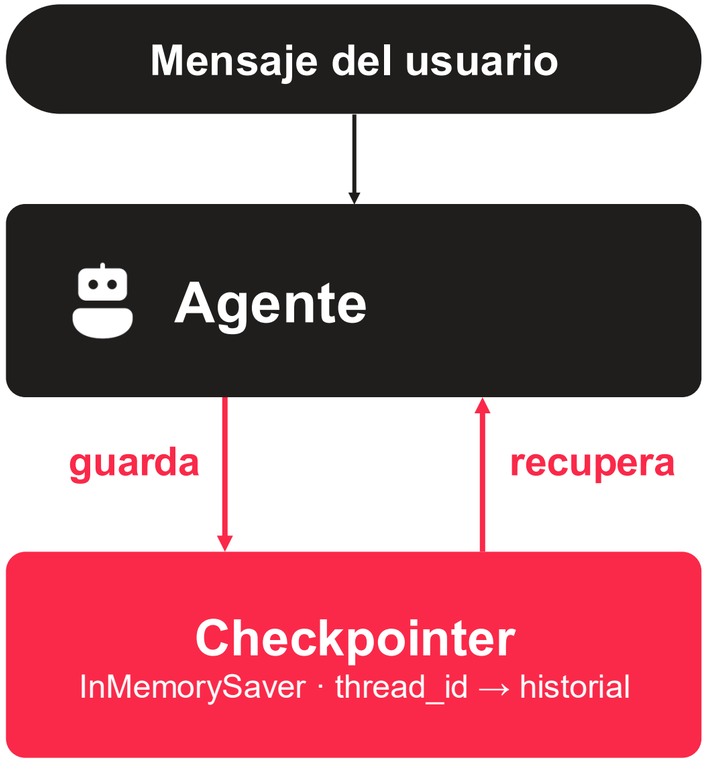
02

Sin checkpointer, el agente olvida

1. Cada invocación de un agente es, por defecto, independiente de las anteriores

2. Un checkpointer (p. ej. InMemorySaver) guarda el historial de mensajes entre invocaciones, asociado a un thread_id

3. En la siguiente invocación con el mismo thread_id, LangGraph recupera ese historial automáticamente



Mismo thread_id → mismo historial

Real: mismo agente, con y sin memoria

Sin memoria

cada turno, un agente nuevo

Hola, me llamo Sebastian.

¡Hola, Sebastian! ¿En qué puedo ayudarte?

¿Cuál es mi nombre?

No lo sé. ¿Cómo te llamas?

Con memoria

mismo thread_id

Hola, me llamo Sebastian.

¡Hola, Sebastian! ¿En qué puedo ayudarte?

¿Cuál es mi nombre?

Te llamas Sebastian.

Misma pregunta, distinto resultado: el checkpointer conserva el historial entre turnos.

Cada thread_id es una conversación aislada

Mismo agente + mismo checkpointer (InMemorySaver)

thread_id: conversacion-1

Sabe que es Sebastian

¿Cuál es mi nombre?

Te llamas Sebastian.

thread_id: conversacion-2

No sabe el nombre

¿Cuál es mi nombre?

No lo sé.

1

El mismo agente y el mismo checkpointer pueden sostener muchas conversaciones en paralelo

2

Esto permite que una aplicación (p. ej. un chatbot de soporte) atienda a muchos usuarios distintos sin mezclar sus historiales

3

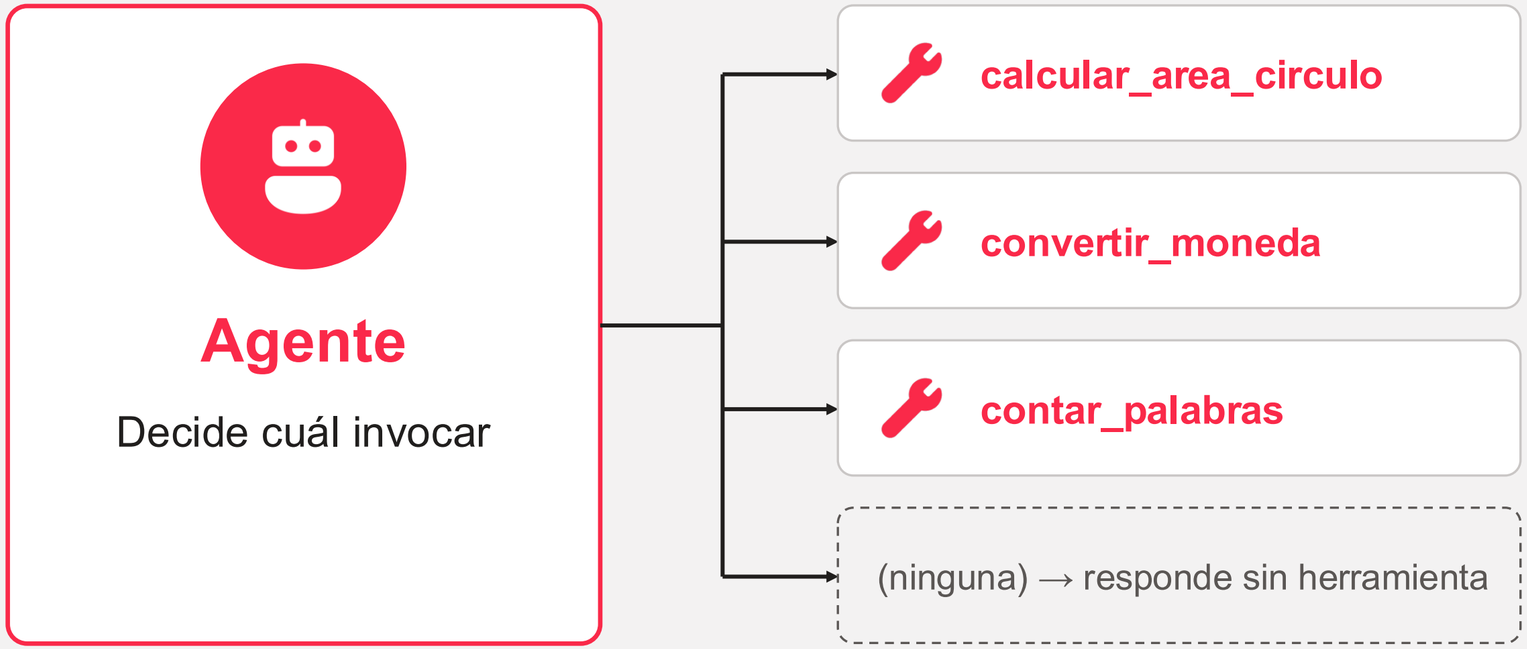
En producción se usa un checkpointer persistente (disco o base de datos) en vez de InMemorySaver, para no perder el historial al reiniciar el proceso

03

Múltiples Herramientas

03

¿Cómo elige el agente entre varias herramientas?



El LLM decide según:

1. **Nombre**
   qué función es
2. **Docstring**
   qué hace y cuándo usarla
3. **Argumentos**
   qué datos necesita

El LLM decide, a partir del nombre, el docstring y los argumentos de cada herramienta, cuál (si hay alguna) usar. No hay un if/else programado. Un docstring claro y específico es lo que permite que esa elección sea correcta.

Resultados reales de enrutamiento

Pregunta	Herramienta invocada	Respuesta
Convierte 100 USD a COP (tasa 4000)	**convertir_moneda**	400,000 pesos colombianos
¿Cuántas palabras tiene la frase...?	**contar_palabras**	6 palabras
Área de un círculo de radio 3	**calcular_area_circulo**	≈28.27
¿Qué hora es en Bogotá?	(ninguna)	"No tengo acceso a esa información"

3/3

herramientas correctas

1/1

rechazo correcto

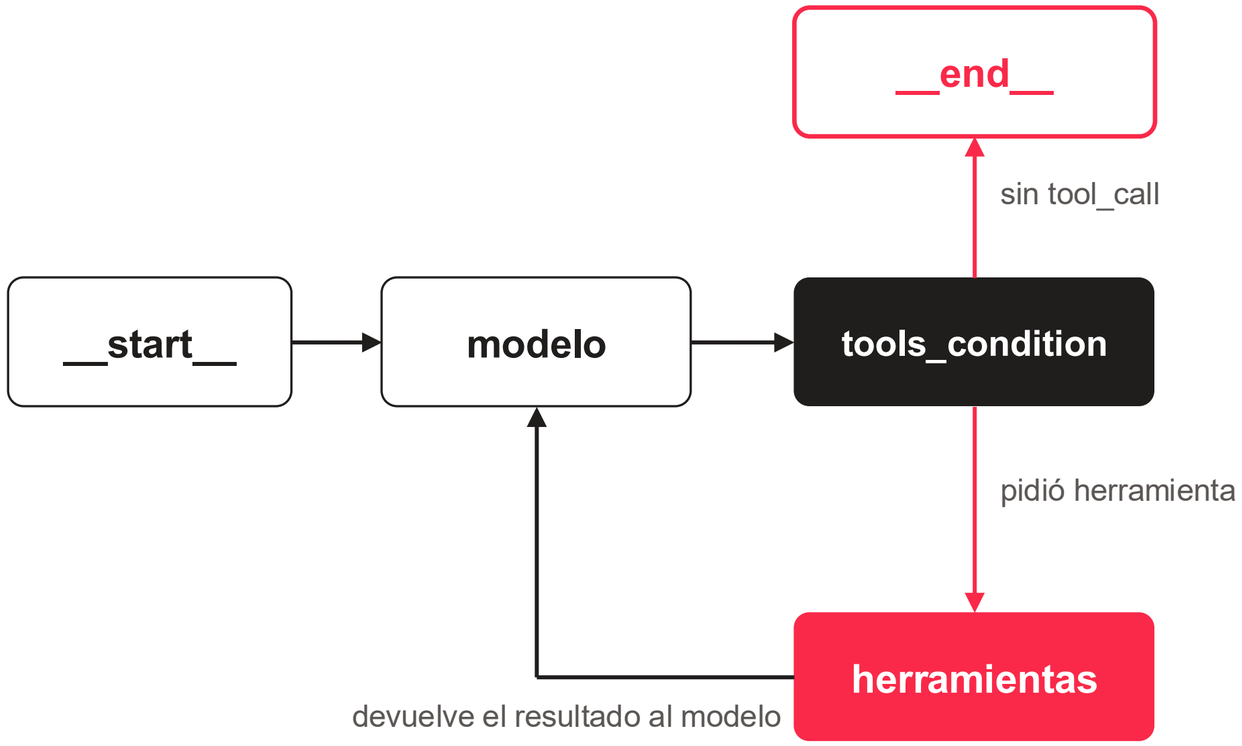
En los tres primeros casos, el agente invocó exactamente la herramienta correcta con los argumentos correctos. En el cuarto, correctamente reconoció que ninguna herramienta disponible podía responder, en vez de inventar una hora.

04

Grafos con LangGraph

04

Un agente es, por dentro, un grafo



Aristas condicionales en rojo

Estado

Los datos compartidos entre nodos, aquí, la lista de messages

Nodos

Funciones que reciben el estado y devuelven una actualización. Modelo (llama al LLM) y herramientas (ejecuta la acción pedida)

Aristas condicionales

Deciden a qué nodo ir según el contenido del estado. tools_condition revisa si el LLM pidió una herramienta

create_agent

Construye, internamente, un grafo prácticamente igual a este

05

RAG como Herramienta

05

El flujo de RAG (Retrieval-Augmented Generation)



1 Indexar

El documento se divide en fragmentos, cada uno se convierte en un vector (embedding) que captura su significado

2 Recuperar

Al hacer una pregunta, se recuperan los fragmentos más similares semánticamente (no por coincidencia exacta de palabras)

3 Usar como herramienta

Esos fragmentos se envuelven en una función @tool más. Para el agente, RAG es una herramienta como cualquier otra

Real: sin RAG inventa, con RAG acierta

"¿Cuántos meses de garantía tienen los auriculares NimbusTech Aria-7 y por qué?"

(marca ficticia)

Sin RAG

≈ 3 años

Respuesta inventada: generaliza sin saberlo

Necesito más contexto... en la Unión Europea la garantía legal suele ser de 3 años... [generaliza sin saberlo]

Con RAG

30 meses

Respuesta fundamentada en el contexto recuperado

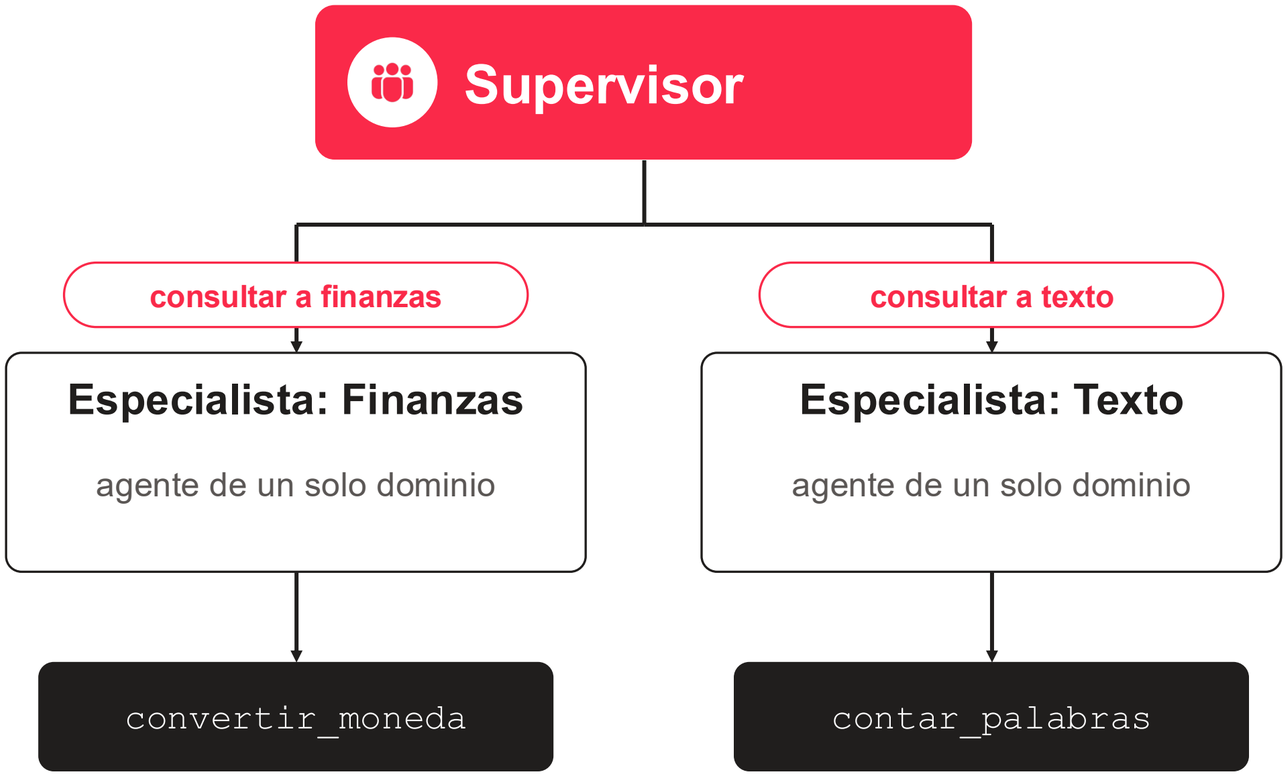
Los auriculares NimbusTech Aria-7 tienen una garantía extendida de 30 meses, debido a su batería de estado sólido.

06

Sistemas Multiagente

06

Patrón: agentes como herramientas



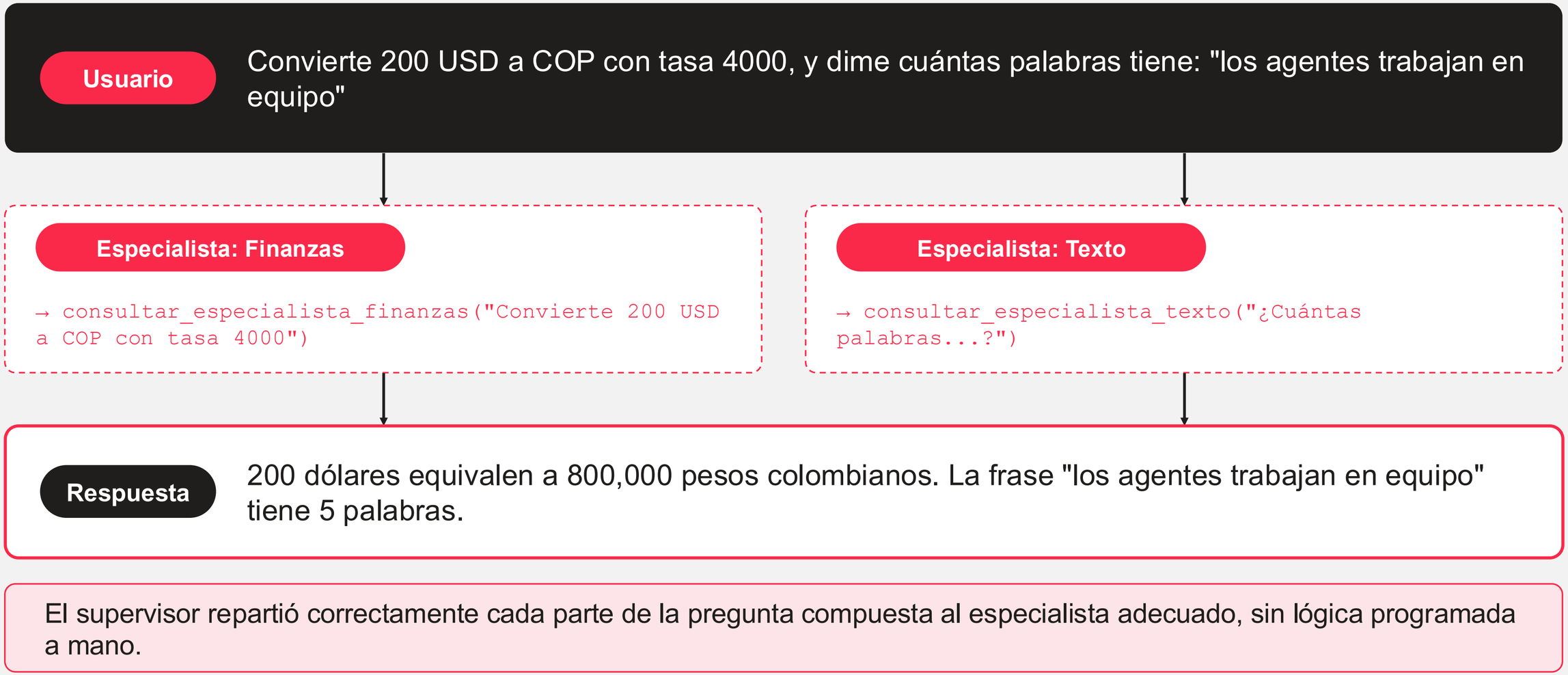
Especialistas enfocados

Cada especialista es un agente con las herramientas de un solo dominio. Más fácil que elija bien entre pocas opciones

Visión acotada

El supervisor no ve las herramientas internas de cada especialista, solo dos herramientas de alto nivel: "consultar a finanzas" y "consultar a texto"

Real: el supervisor delega correctamente



El supervisor repartió correctamente cada parte de la pregunta compuesta al especialista adecuado, sin lógica programada a mano.

07

Evaluación de Agentes

07

Dos dimensiones: trayectoria y resultado

Trayectoria

¿El agente invocó la herramienta correcta (o correctamente ninguna)?

Resultado

¿La respuesta final es correcta?

	Resultado correcto	Resultado incorrecto
Trayectoria correcta	**Ideal** Herramienta correcta y respuesta correcta	**Mal redactado** Elige bien la herramienta, pero redacta mal la respuesta
Trayectoria incorrecta	**Camino equivocado** Llega a la respuesta correcta por el camino equivocado	**Falla total** Ni la herramienta ni la respuesta son correctas

Evaluación determinista (reglas programadas) en vez de "LLM como juez": más simple y barata cuando la respuesta se puede verificar con una regla exacta

Resultado real sobre 4 casos de prueba

100%

Exactitud de trayectoria

100%

Exactitud de resultado

Pregunta	Trayectoria	Resultado
Área de un círculo de radio 6	**OK**	**OK**
Convierte 50 euros a pesos (tasa 4300)	**OK**	**OK**
Palabras en "hola mundo desde el agente"	**OK**	**OK**
¿Qué tiempo hace hoy en Medellín?	**OK (ninguna herramienta)**	**OK**

4 de 4 casos correctos en trayectoria y resultado (incluido el caso en que ninguna herramienta aplica)

RECURSOS

Resumen de la Unidad 5

	Tema	Notebook del curso
01	**Fundamentos de agentes (ReAct)**	`1-fundamentos-agentes.ipynb`
02	**Memoria y estado**	`2-memoria-y-estado.ipynb`
03	**Múltiples herramientas**	`3-multiples-herramientas.ipynb`
04	**Grafos con LangGraph**	`4-grafos-langgraph.ipynb`
05	**RAG como herramienta**	`5-rag-como-herramienta.ipynb`
06	**Sistemas multiagente**	`6-multiagentes.ipynb`
07	**Evaluación de agentes**	`7-evaluacion-de-agentes.ipynb`

¡Gracias!

De razonar y actuar en ciclos, a coordinar equipos de agentes.

